MetroIsJustABus — Requirements and Design Traceability Review

Review date: 10 October 2026
Project brief reviewed: *Automated Software Engineering — Course Project 2026/2027*, version 1.0 dated 29 September 2026
Requirements/design file reviewed: requirements.md
Purpose: audit the assumptions, trace every requirement and proposed design element to the brief or to an explicitly labelled assumption/design decision, identify conflicts and unsupported additions, and recommend corrections.

Contents

1. Executive summary
2. Review method and evidence labels
3. Source map: where the brief provides evidence
4. Audit of the assumptions and actors
5. Functional requirement traceability (FR-1 to FR-25)
6. Non-functional requirement traceability (NFR-1 to NFR-9)
7. Data model audit
8. Component architecture audit
9. Interface and event-catalogue audit
10. Behavioral-model audit
11. Idempotency summary audit
12. Design decisions and trade-offs audit
13. Open questions
14. Important brief requirements missing or not made explicit
15. Recommended revised assumptions/decisions register
16. Prioritized action list
17. Final assessment

Appendix A. Compact traceability index

1. Executive summary

The requirements file is a strong first-pass design: it covers most of the domain described in the brief, recognizes idempotency and fault isolation, proposes useful models and interfaces, and identifies important open questions. However, it is **not yet safe to treat every item as a confirmed project requirement**. Some details are faithful paraphrases of the brief; others are reasonable inferences; and several are new scope, performance targets, or business rules that are not stated in the brief.

The most important findings are:

1. **Architecture conflict — resolve before implementation.** The brief requires a distributed, multi-component system with independently deployable supporting

components and a notification capability across an independent application boundary. The requirements file later says the system "can be deployed as a modular monolith first and split later." If that means that all components, including notifications, ship as one deployable, it conflicts with the brief. Keep modular boundaries inside the core if useful, but deploy the notification capability separately at minimum.

2. **Quantitative NFRs are invented targets, not extracted requirements.** The targets of 99.9% availability, validation p95 under 300 ms, and cached schedule-read p95 under 200 ms are unsupported by the brief. They may be adopted as team-proposed targets, but must be labelled and justified as such, or removed until workload and infrastructure are known.
3. **Some ticket rules add unconfirmed business policy.** Transfer windows, activation on first use, anti-passback intervals, refunds, cancellation, suspension, and specific ticket kinds are not defined in the brief. They can be options in the model, but must not be represented as agreed requirements without a decision from the project stakeholders/teacher.
4. **Several assumptions have real scope or correctness implications.** External payment integration, offline validation, precise staff roles, GPS being out of scope, notification channels, holiday calendars, and manual/sensor occupancy corrections are not fully specified by the brief. Most are plausible, but they need explicit labels and, where material, confirmation.
5. **There are internal model inconsistencies.** Vehicle type is listed as a stable entity but no corresponding entity is defined; the relationship Trip 1..1 OccupancySnapshot conflicts with the snapshot's optional stop sequence and update timestamp if multiple snapshots are intended; and the validation sequence does not clearly distinguish persisted successful validations from persisted rejected attempts.
6. **Important course-level requirements are only partially represented.** The requirements file describes the domain well, but the brief also mandates synchronous APIs, asynchronous/event-based communication, explicit interface/data contracts, timeouts and communication-failure handling, automated component/system validation, versioned artifacts, deployment traceability, a self-hosted CI/CD runner in development, separate reproducible development and production environments, and evidence about AI/agent use throughout the lifecycle. These should be tracked explicitly, even if they are not passenger-facing functional requirements.

Overall verdict

- **Core domain coverage:** good.
- **Traceability of FR-1 to FR-25:** mostly strong, with several requirements partially supported or extended beyond the source.
- **Traceability of NFR-1 to NFR-9:** mixed; NFR-1 through NFR-4 are broadly motivated by the brief but contain implementation choices; NFR-5 is unsupported as written; NFR-6 and NFR-8 are largely added quality goals; NFR-7 and NFR-9 are partly supported.
- **Architecture compliance:** a material conflict must be fixed.

- **Readiness:** suitable as a working proposal, but clearly separate confirmed requirements from assumptions and design choices before submitting it as a requirements specification.

2. Review method and evidence labels

This review uses the following labels:

- **E — Explicit:** the brief directly states the behavior or constraint.
- **P — Partial / inferred:** the brief supports the general idea, but the requirement adds detail, scope, or a particular interpretation.
- **A — Assumption / unsupported addition:** the brief does not establish the claim. It can still be adopted if the team labels it as an assumption or proposed decision.
- **D — Design choice:** a possible implementation/model/interface chosen by the team rather than a requirement imposed by the brief.
- **C — Conflict / inconsistency:** the item contradicts another source requirement or another part of the current requirements file.

A design choice is not automatically bad. The issue is whether the document accurately represents its status. For example, choosing REST, an event bus, an outbox pattern, or a database uniqueness constraint can be sensible without being stated in the brief. Those choices should be documented as the team's design, not cited as if the brief mandated their exact form.

3. Source map: where the brief provides evidence

Use these source locations throughout this review. Page numbers refer to the uploaded version 1.0 PDF.

- **B1 — Domain behavior.** *Location:* Technical Description, pp. 7–8. *Evidence:* lines/stops and ordered sequences; schedules and trips; day/calendar variation; disruptions; tickets and ticket validation; occupancy/capacity; accounts/subscriptions/notifications; operational staff; history; resilience and duplicate-operation handling.
- **B2 — Architecture.** *Location:* Application and Architecture, p. 8. *Evidence:* distributed multi-component system; core information system; independently deployable supporting component(s); passenger-facing web application; independent notification boundary; explicitly defined APIs/messages/events; a single backend REST application is insufficient.
- **B3 — Communication.** *Location:* Communication and Integration, p. 9. *Evidence:* synchronous APIs and asynchronous/event-based communication; interface/data definitions; handling failures, timeouts, unavailability, retries, duplicates, and cross-component consistency; justify protocols and data formats.
- **B4 — Verification.** *Location:* Testing and Verification, p. 9; Project Phases, p. 3. *Evidence:* automated unit, integration, system/end-to-end tests; interface/inter-component verification; automated quality/static analysis; failure/recovery tests; other suitable techniques. Acceptance testing is also named in the Testing & Verification project phase on p. 3.

- **B5 — CI/CDE/CD.** *Location:* CI/CDE/CD, p. 9; milestones, pp. 4–5. *Evidence:* automated build/test/quality checks; versioned deployable artifacts; integration/system validation; automated deployment to dev/prod; gates before production; source-artifact-deployed-version traceability; milestone pipeline requirements.
- **B6 — Infrastructure and operations.** *Location:* Deployment and Infrastructure, pp. 9–10; Project Phases, p. 3. *Evidence:* separate dev and production environments; self-hosted CI/CD runner and integration/system tests in dev; production hosting; reproducible, version-controlled infrastructure; containers; configuration/secrets/persistent data/networks/dependencies; deployment validation and recovery; monitoring and observability.
- **B7 — AI-assisted engineering and evidence.** *Location:* AI-Assisted and Agentic Engineering, p. 10; Introduction/Objectives/Phases, pp. 1–3; final report, pp. 4–5. *Evidence:* use LLMs/agents throughout requirements, design, coding, tests, integration, deployment and other appropriate activities; provide context and constraints; validate output and know when humans intervene; collect evidence on delegated tasks, intervention, validation, failures, rework and effects.
- **B8 — Course organization and deliverables.** *Location:* Introduction/Objectives/Project Structure/Phases/Milestones/Evaluation, pp. 1–6. *Evidence:* five-student teams; milestone dates and deliverables; final system, report, evidence/metrics, 10-minute demo, peer/self-evaluation and individual defenses; graded engineering process, technical delivery, measurement/critical analysis, reporting/demo. These are project obligations, not all domain-system requirements.

4. Audit of the assumptions and actors in `requirements.md`

4.1 Actor list

- **Passenger (anonymous or registered). Status: P.** Passengers browse network information; the brief says passengers may create accounts and subscribe (B1, p. 7). It does not explicitly say which pages are accessible anonymously or whether accounts are optional for purchases. **Recommendation:** keep anonymous browsing as a likely default, but label it an assumption. Specify that account ownership is required for account-associated tickets and personalized subscriptions only if that is the chosen policy. Confirm whether ticket purchase requires login.
- **Operational staff. Status: E.** Staff maintain network/schedule information, manage ticket-related information, register/manage disruptions, and need ticket-usage and occupancy information (B1, pp. 7–8). **Recommendation:** keep; this is explicitly in the brief.
- **Planner, dispatcher/traffic controller, ticketing manager as separate roles. Status: A/P.** The brief defines “operational staff” but does not name these subroles or require different permission sets. **Recommendation:** treat these as reasonable examples, not confirmed requirements. Start with an operational-staff role and define narrower roles only when responsibility separation is needed or confirmed. If role-based access control is adopted, call it a security/design requirement, not a direct quotation from the brief.
- **Validator device. Status: P/A.** Ticket validation is explicitly required, but physical validator devices and their location (on-board or at-stop) are not defined (B1, p. 8).

Recommendation: a validator client/API is a reasonable system boundary. Device form and physical location are design assumptions. Model a validation client abstractly until the implementation is agreed.

- **Notification channels (email/push/SMS), external providers. Status: P/A.** Notifications and user preferences are required (B1, p. 8), but channels and provider integrations are unspecified. **Recommendation:** choose one implementable channel or a mock adapter unless multi-channel delivery is explicitly expected. Keep the channel/provider list as an assumption. "Multiple channel support" is not mandated by the brief.
- **External payment provider. Status: A.** The brief says passengers acquire tickets and ticket types may have prices; it does not mandate an external payment gateway or real-money integration (B1, p. 8). **Recommendation:** do not make a third-party payment service a mandatory dependency without confirmation. A simulated payment adapter may be sufficient for a course prototype. If external payments are chosen, define success/failure, idempotency, and no-ticket-without-confirmed-payment behavior as design decisions.

4.2 The five assumptions currently listed

Assumption A1 — "Validation happens on-board or at stops via devices that submit validation events. They may retry and may be briefly offline."

- **Supported part:** validation itself, the need to assess validity, and communication-failure handling are explicit (B1, p. 8; B3, p. 9). The brief explicitly warns against duplicate validations caused by communication failures (B1, p. 8), so retry-safe operations are well grounded.
- **Not established:** physical location of validators and acceptance of validations while fully offline.
- **Risk:** truly offline acceptance can conflict with checking the current ticket state, previous validations, and permitted journey count. Two disconnected devices could accept the same limited-use ticket before they synchronize. A retry after a lost response is different from allowing new validation while offline.
- **Recommendation:** narrow the assumption to: *"A validator client submits validation events. If a response is lost or connectivity is interrupted, the client may retry the same event using a stable event ID. Unless offline policy is explicitly approved, validity is checked against the authoritative online state; new validations are not accepted offline."* If offline acceptance is intended, define offline limits, local state, conflict resolution, and reconciliation explicitly.

Assumption A2 — "Occupancy is estimated from validations (boardings) and, optionally, from staff or sensor corrections. It is an estimate, not ground truth."

- **Supported part:** trip occupancy and available capacity must be monitored; passenger activity, including ticket validations, may affect occupancy; operational staff must have capacity information, with passenger exposure where appropriate (B1, p. 8).
- **Not established:** each accepted validation equals one boarding; sensors exist; staff corrections are available; the exact occupancy formula or granularity.

- **Risk:** a validation is only a proxy for boarding. Transfers, failed/no-ticket boarding, re-validation rules, alighting, or validation without boarding can make the estimate diverge from actual occupancy. The brief does not ask for sensor integration.
- **Recommendation:** retain the estimate caveat. A suitable initial assumption is *"Accepted validations are one input to an estimated occupancy measure; the system does not claim ground-truth occupancy. Manual corrections and sensors are optional extensions."* Decide whether the output is a whole-trip count or per-stop-segment estimate before implementing the stop_sequence field.

Assumption A3 — "Payment is delegated to an external provider."

- **Support:** tickets have prices and passengers can acquire tickets (B1, p. 8), but this does not establish payment-provider integration.
- **Status: A** — an unsupported implementation/scope assumption.
- **Recommendation:** replace with *"The project will implement ticket acquisition; the payment mechanism is unresolved. Unless a provider is specified, use a simulated payment adapter and isolate it behind an interface."* If real external payment is selected, document the provider and failure/idempotency behavior as a separate decision.

Assumption A4 — "Real-time vehicle positions are out of scope. 'Delay' is entered or updated by staff."

- **Support:** the brief requires disruption management, including delays, and says staff register and manage disruptions (B1, p. 8). It does not require GPS/vehicle telemetry, but it also does not explicitly exclude it or prescribe how delay values are obtained.
- **Status: P/A.** A no-GPS first version is a reasonable scope boundary; manual delay entry is a proposed operating assumption.
- **Recommendation:** say *"No live vehicle-position integration is assumed in the initial scope because the brief does not specify a positioning data source. Delay information will be recorded by operational staff unless another source is approved."* This makes it clear that the source of delay data remains an assumption.

Assumption A5 — "Roles for staff (planner, dispatcher, ticket manager) are separate permissions."

- **Support:** operational staff have several responsibilities (B1, p. 8); separate permissions are not explicitly required.
- **Status: A/P.** Sensible security design, but not a source-stated requirement.
- **Risk:** can create unnecessary scope and a role matrix before the actual responsibilities are agreed.
- **Recommendation:** define a base staff authorization rule first. Add separate planner/dispatcher/ticket-manager roles only if they enforce a real separation of duties or the teacher confirms that level of access control.

4.3 Other assumptions currently embedded outside section 1

The following assumptions appear in requirements, data, interfaces, and behavior even though they are not listed in the assumptions section. They should be moved into a clearly labelled "Assumptions and design decisions" register.

1. **Date filtering and specific service-calendar classes (weekday/weekend/holiday/special):** schedule variation by weekday or other operating conditions is explicit (B1, p. 7); date-based filter behavior and exact calendar categories are implementation choices.
2. **Line variants and direction:** an ordered stop sequence is explicit (B1, p. 7); representing direction and variants as separate LineVariant records is a design choice.
3. **Retiring a line/stop and effective dates for every network change:** staff-maintained network/schedules are explicit; "retire" and a required effective-from date for each entity are not stated at that level of detail.
4. **Stop closure as a disruption type, alternative-stop sets, partial cancellations, and exact disruption target combinations:** broad disruption kinds and scopes are explicit; these specific representations are implementation assumptions.
5. **Ticket kinds SINGLE, MULTI, PASS, TIMED; transfer windows; on-sale switch; ticket activation on first use; expiry boundary; anti-passback interval; refunds, staff cancellation, and suspension:** no such detailed business rules are given in the brief. Treat them as proposed examples pending decisions.
6. **Exactly-at-most-once notification semantics defined by (source_event_id, subscription_id, channel):** duplicate notifications are explicitly to be avoided (B1, p. 8); this particular deduplication key and meaning of "once" are design choices. External-provider delivery guarantees need separate treatment.
7. **Numeric occupancy for staff and labels low/medium/high/full for passengers:** staff capacity information and passenger availability "where appropriate" are explicit, but exact presentation and thresholds are not.
8. **Passenger-visible notification history:** preserving historical information is generally supported; a dedicated passenger-facing notification-history page is not expressly required.
9. **A staff audit log with actor, before/after state, and timestamps:** traceability is sensible, but those exact fields and auditing every staff change are not specified in the domain description.
10. **NFR figures, p95 targets, expected peak-load pattern, circuit breakers, caches, dead-letter queues, an outbox, schema versions, correlation IDs, REST path names, and specific API response codes:** these are targets or design mechanisms, not requirements directly quoted from the brief.
11. **GDPR-specific retention/deletion and "notification content free of sensitive data":** these are prudent privacy considerations, but are not stated in the course brief. The team should determine applicable legal/institutional obligations separately and reconcile deletion with historical/audit needs; this review does not claim the brief itself establishes GDPR rules.
12. **"Vehicle type" in the lifecycle table:** a vehicle-type entity is not otherwise defined in the requirements file and is not required by the supplied domain description.

5. Functional requirement traceability (FR-1 to FR-25)

Each entry below restates the requirement's substantive content, identifies the evidence, and says whether the current formulation is faithful or needs qualification. "Evidence" refers to B1–B8 in Section 3.

Network and schedules

FR-1 — Browse lines, stops, and each line's ordered stop sequence

Status: E. B1, p. 7 says a line defines an ordered sequence of stops and passengers can consult the network, lines, and stops. Showing the sequence is directly supported. The particular endpoint GET /lines/{id}/stops is a design proposal, not a brief requirement.

FR-2 — View schedules per line and stop, filter by date, include weekday/weekend/holiday/special variants

Status: P. B1, p. 7 explicitly says published schedules define planned trips/times and may vary by day of week or other operational conditions. Per-line/per-stop schedule views follow from passenger consultation of schedules. Date filtering and the exact list of calendar categories are reasonable refinements, not exact brief wording. Keep the feature but label the calendar representation as a design assumption and avoid implying the brief names these exact variants.

FR-3 — View scheduled trips and expected times at each stop

Status: E. B1, p. 7 describes planned trips for each line and expected times at each stop, and says passengers can consult scheduled trips. Directly supported.

FR-4 — Passenger-facing information is the effective view of scheduled data combined with active disruptions

Status: E/P. B1, pp. 7–8 requires information to account for known operating changes and disruptions to be reflected in passenger information. Combining schedule and disruption information into an "effective view" is a faithful interpretation. Computing that view at read time or via a projection/cache is a design choice, not mandated.

FR-5 — Staff can create, update, and retire lines, stops, line-stop sequences, schedules, and trips; every change takes effect from a stated date

Status: P. Staff maintenance of network and schedule information is explicit (B1, p. 8). Line/stop/sequence/schedule/trip CRUD is a reasonable decomposition of that duty. "Retire" and an effective-from date for every kind of change are not explicit. Effective dates and history are useful for avoiding retroactive changes, but the exact rule should be recorded as a design assumption or confirmed requirement. Consider whether trips are manually edited or generated from schedules; the current data model assumes they can be generated.

Disruptions

FR-6 — Staff register delay, cancellation, diversion, stop-closure, or other disruptions

Status: E/P. Delay, cancellation, diversion, and other disruptions are explicit in B1, p. 7; staff registration/management is explicit on p. 8. A distinct "stop closure" type is a reasonable interpretation of a disruption affecting an individual stop, but is not named verbatim. Keep "stop closure" as a proposed subtype.

FR-7 — Each disruption has a scope (line, stop, trip, or line+stop), period and status; staff can update/close it

Status: P. Disruptions affecting a line, stop, particular trip, or period are explicit (B1, p. 7), as is staff management (p. 8). The combined line+stop scope, explicit status field, update/close workflow and chosen lifecycle states are design refinements. They make sense, but should be labelled as a model, not all as direct brief language.

FR-8 — Delays carry a magnitude, diversions an alternative stop set, and cancellations can be complete or partial

Status: P/A. A delay, diversion, and cancellation are explicit types (B1, p. 7), but the specific stored data and semantics are not. A magnitude is practically required to represent a delay; alternative stops and partial cancellation are plausible, but need definitions. State these as proposed data/behavior rules. In particular, confirm what a diversion's "alternative stop set" means and whether stop order, omitted stops, added stops, and changed arrival times must be represented.

FR-9 — Passenger views reflect disruption creation, updates, and ending

Status: E/P. The brief says disruptions are reflected in passenger information (B1, p. 7), staff can manage them (p. 8), and changes may affect information already in passenger use and trigger notifications. Updating and ending a disruption must affect the effective view. Separate "created/updated/closed" events are a design choice.

Tickets

FR-10 — Staff manage ticket types, including price, validity period, journey count, transfer-window rules, and whether a type is on sale

Status: P. Staff manage ticket-related information, and ticket types may vary by price, validity period, conditions of use, and permitted journeys (B1, p. 8). Price, validity, conditions and journey count are directly supported. "Transfer window" is one possible condition, not specified by the brief; an on-sale flag is a useful implementation choice, not explicit. Ticket-type maintenance is supported at a general level.

FR-11 — A registered passenger can buy a ticket and view account tickets, state and remaining validity/journeys

Status: P. Acquiring tickets and consulting tickets associated with an account are explicit (B1, p. 8). Ticket state and remaining validity/journeys follow from validity depending on current

state, time, previous validations and allowed journeys. The “registered passenger” restriction is a reasonable assumption but the brief does not explicitly state that registration must be mandatory for every acquisition. Confirm the account/purchase policy.

FR-12 — Ticket purchase is idempotent so a retried request cannot issue two tickets

Status: E/P. The brief explicitly says communication failures must not cause the same ticket to be issued multiple times (B1, p. 8). Idempotent purchase is the right interpretation. An HTTP idempotency header and unique database key are implementation choices, not required techniques.

Validation

FR-13 — Check a validation against ticket type, state, prior validations and time; return accepted/rejected and a reason code

Status: E/P. Ticket validity depending on ticket type, current state, previous validations and time is explicit; the system must decide whether attempted use is valid (B1, p. 8). Accepted/rejected is a natural result representation. A machine-readable “reason code” is a design detail, although it is useful for clients and tests.

FR-14 — Record accepted validations and update ticket state (journeys remaining, activation, exhaustion)

Status: E/P. Recording successful validations is explicit (B1, p. 8). Updating the remaining permitted journeys/current state is a reasonable consequence of ticket validity rules and permitted journey counts. Activation and exhaustion are specific lifecycle states not named in the brief; keep them as proposed model states until ticket rules are agreed. The document should make clear whether an accepted validation and the ticket state change must be atomic; strong transactional handling is an architecture decision that addresses correctness.

FR-15 — Validation is idempotent per event; a retry returns the first result without a second record

Status: E/P. Avoiding duplicate validation records caused by communication failures is explicit (B1, p. 8). Stable event identifiers, unique constraints and returning the stored result are design mechanisms. Be explicit about whether the result of a rejected attempt is also stored and replayed: the brief specifically requires recording *successful* validations, not necessarily every rejected attempt.

FR-16 — Staff can query ticket usage/validation activity by ticket, line, trip, stop and period

Status: P. The brief requires operational staff to obtain information about ticket usage and validation activity (B1, p. 8). The five query filters are a design refinement. They are plausible if the validation records contain those references and the corresponding data is available, but each filter is not separately mandated. Confirm what usage reporting the staff role needs.

Occupancy and capacity

FR-17 — Every scheduled trip has capacity and an occupancy estimate updated by validations

Status: P. Monitoring occupancy of scheduled trips and available capacity is explicit, and passenger activity including validations may affect occupancy (B1, p. 8). The exact rule "every trip has a capacity" and using each validation as a direct update are refinements. The existing assumption that occupancy is an estimate should remain visible; describe which events affect the estimate and how corrections/rebuilds work.

FR-18 — Staff see numeric occupancy; passengers may see low/medium/high/full when enabled

Status: P. Capacity information must be available to operational staff and, where appropriate, to passengers (B1, p. 8). Numeric values for staff and four passenger labels are proposed presentation rules. "Where enabled" is an appropriate optional boundary, but thresholds and the meaning of "full" need a decision. Do not represent these exact categories as source-mandated.

Accounts, subscriptions, notifications

FR-19 — Passengers can register and authenticate

Status: P. The brief says passengers may create accounts (B1, p. 8), which supports registration/account creation. Authentication is implicit in private account access and managing preferences, but the method and exact security requirements are not given. Account authentication is a reasonable enabling requirement.

FR-20 — Subscribe to a line, stop, or trip, optionally with change-type/time-window filters

Status: E/P. The line, stop, and scheduled-trip examples are stated in B1, p. 8. Optional filters for change types and time windows are not stated; they are a reasonable extension but add data/model complexity. Make them optional design scope unless explicitly needed.

FR-21 — Set channel preferences and quiet hours, and pause/cancel subscriptions

Status: P. Passengers must control subscriptions and notification preferences (B1, p. 8). Channel preferences are within the normal meaning of preferences. Quiet hours and the specific pause/cancel actions are detailed product decisions. The document should not claim quiet hours are mandated; confirm whether the project will implement them. Define whether quiet hours defer notifications or suppress them.

FR-22 — For a disruption/schedule change, identify affected subscribers and notify at most once per event, subscription, channel

Status: E/P. Finding affected passengers and notifying them when relevant information changes is explicit (B1, p. 8). Schedule changes are supported by the wording "something relevant changes" and the statements that staff changes can affect passenger information and generate notifications. Duplicate notification prevention is explicit (B1, p. 8). The exact dedup key and "once per event, subscription, and channel" grouping are design semantics. Clarify how a provider response timeout is handled: retry-safe internal queuing is necessary, but exactly-

once external delivery cannot be guaranteed unless the channel provider supports an appropriate idempotency mechanism.

FR-23 — Passengers can view notification history

Status: P/A. The brief says historical information may be relevant and names notifications among changing/historical information (B1, pp. 7–8). However, it does not explicitly require a passenger-facing notification-history view. FR-23 is a reasonable product feature and aligns with the current Notification model, but should be labelled an assumption or removed if scope is tight. Note that FR-25's historical-data statement does not automatically require a user interface for passengers to browse that history.

Staff operations and history

FR-24 — Staff changes are authorized by role and audited with who/what/when

Status: P/A. Staff maintenance/management operations are explicit (B1, p. 8), but per-role authorization and the specified audit record are not stated in the domain brief. Treat this as a proposed security/audit requirement. It is a good candidate for a separate, confirmed requirement, especially if more than one staff role is introduced. If adopted, define coverage of changes, whether before/after values are stored, and who may view audit data.

FR-25 — Retain and query historical schedules, trips, disruptions, validations and notifications

Status: P. The brief explicitly says that historical information may be relevant for understanding previous system activity and operation (B1, p. 8). The entity list is a sensible interpretation. Exact retention periods, query capabilities, archival method and immutability are not specified. Preserve history where it is needed to explain past service/ticket activity, but set retention and deletion rules deliberately rather than assuming indefinite retention. Reconcile this with any privacy obligations separately.

6. Non-functional requirement traceability (NFR-1 to NFR-9)

NFR-1 — Availability: read paths stay available if notification, ticketing or occupancy is down; target at least 99.9% for reads

Status: P/A; numeric target unsupported. The brief says a temporary failure in an individual part should not unnecessarily make the whole system unavailable, and explicitly gives notification failure not preventing lines/stops/schedules or unrelated functionality as an example (B1, p. 8). Extending this to ticketing/occupancy is a reasonable system-level target. The **99.9%** value has no support in the brief. Remove it or label it as a team-proposed service-level objective and explain the measurement window, which reads count, and how it will be tested.

NFR-2 — Fault isolation with timeouts, circuit breakers, and asynchronous integration

Status: P/D. The required outcome—temporary component failures should not unnecessarily disable other functions and communication failures should not produce unintended effects—is explicit (B1, p. 8; B3, p. 9). Timeouts are explicitly within the communication requirements

(B3); circuit breakers and asynchronous integration are solution mechanisms/choices. The brief separately requires asynchronous/event-based communication (B3). Keep the desired outcome as the requirement and list the mechanisms as the proposed design, with tests showing their effect.

NFR-3 — Idempotency: purchase, validation and notification delivery are safe to retry

Status: E/P. The brief explicitly names duplicate ticket issuance, duplicate validation records, and duplicate notifications as effects that communication-failure handling must prevent (B1, p. 8). “Purchase” is a plausible interpretation of duplicate ticket issuance. The specific safe-retry contracts and database keys are design choices. For external notification delivery, make the guarantee precise and provider-dependent rather than promising a mathematical exactly-once effect without support from the channel.

NFR-4 — Strong consistency for ticket state/validation; eventual consistency for occupancy, notifications and read models

Status: P/D. The brief requires ticket validity decisions based on current state/previous validations and requires handling duplicates and cross-component consistency (B1, p. 8; B3, p. 9). Strong consistency for ticket/validation operations is a sensible design response. The brief does not prescribe the named consistency models. Occupancy and notifications being eventually consistent fits an asynchronous design, but record it as a design decision and state acceptable staleness/recovery behavior where useful.

NFR-5 — Validation p95 under 300 ms; schedule reads p95 under 200 ms (cached); peak load is validation and morning schedule reads

Status: A. None of these exact latency limits, percentile measures, caching requirements, or peak-load assumptions appear in the brief. The brief does call for performance/security testing where appropriate (B4) but supplies no service-level figures. **Recommended action:** remove this as a “requirement from the brief.” Either convert it to a clearly marked proposed target after selecting an environment and test load, or replace it with a measurable requirement established by the team and approved by the teacher/stakeholders. Do not assume a morning peak unless there is evidence for that workload.

NFR-6 — Security and privacy: RBAC, data minimization, no sensitive notification content, GDPR-compliant retention/deletion

Status: A/P. Accounts, subscriptions, tickets and staff actions necessarily involve access control considerations, but the technical description does not state these four specific controls or cite GDPR. These are prudent topics for design and may be required by applicable law or institutional policy, but that needs a separate validation; it is not traceable to this project statement as written. The blanket “GDPR-compliant retention and deletion” wording is too broad without defining data categories, lawful retention, deletion/erasure behavior, and how it coexists with historical/audit records. Keep the controls as proposed security/privacy decisions and confirm applicable requirements independently.

NFR-7 — Auditability: ticket, validation and staff changes are traceable

Status: P. Successful validation records and staff access to ticket-usage/validation information are explicit (B1, p. 8); the relevance of historical information is also explicit. Auditing all staff changes and every ticket lifecycle change is a reasonable extension, not stated explicitly. Define the audit scope and event details if retained.

NFR-8 — Evolvability: new ticket types, notification channels and disruption types can be added without changing core logic

Status: A/D. The brief defines more than one ticket/disruption type and expects system-level evolution and design justification generally (B1, p. 7; B7). It does not require a plug-in strategy or prohibit changes to core code when a type is added. This is an architectural quality goal, not an extracted requirement. Keep it if the team values it, but avoid overengineering a strategy interface before concrete variation is known.

NFR-9 — Observability: metrics, structured logs and correlation IDs across components

Status: P/D. Monitoring, observability, diagnosis of failures, and validation of deployed behavior are required at course/system level (B6, p. 3 and pp. 9–10). The specific forms—structured logs and cross-component correlation IDs—are implementation choices that would support the requirement. Metrics are also needed for the course's evidence/measurement work (B7/B8), but distinguish operational system metrics from AI-process metrics.

7. Data model audit

The data model is a proposed implementation design, not a list of items all explicitly specified by the brief. The core concepts are traceable; many fields and cardinalities are design choices. The field-level remarks below cover the original model and its lifecycle table.

7.1 Lifecycle table

- **Stable: Line, Stop, LineStop, TicketType, Vehicle type.** Lines/stops and their ordered relationship are explicit (B1, p. 7). Versioning with effective dates is a useful design response to schedule variation and historical needs, not a direct requirement for every stable entity. Vehicle type is unsupported and is not defined in the schema; remove it unless a vehicle-specific feature is added. TicketType is not necessarily "stable" because the model proposes versioning and on-sale changes; keep historical ticket-type versions if sold tickets must retain the rules they were bought under.
- **Planned: ServiceCalendar, ScheduleVersion, ScheduledTrip, StopTime.** Schedules, their variation, scheduled trips, and expected times at stops are explicit (B1, p. 7). Calendar records, schedule versioning, trip materialization per service date, and generation for a date range are design choices. The model should clarify whether a trip is pre-generated and persisted or computed from a schedule.
- **Operational: Disruption, Occupancy.** Both concepts are explicit (B1, pp. 7–8). Status lifecycle and time bounds are modeling choices that fit the source.
- **Transactional: Ticket, Purchase, Validation.** Tickets, acquisition, validation and recording successful validations are explicit (B1, p. 8). Treating purchase/validation

records as append-only or immutable after completion is a design choice. Avoid implying every rejected validation must be retained unless that is chosen.

- **Interaction: Account, Subscription, Notification.** Accounts, subscriptions, notification preferences and notifications are explicit (B1, p. 8). The exact mutable/append-only split is a design choice.

7.2 Entity/attribute review

Line(id, code, name, status)

- **Source-backed concept:** line and line identity/network information (B1, p. 7).
- **Design fields:** id, code, name, status; the brief does not prescribe names or a status field.
- **Review:** reasonable. Clarify what status means (active/retired, published/unpublished, or something else) and avoid using one field for multiple lifecycle concepts.

Stop(id, name, lat, lon, status)

- **Source-backed concept:** stops (B1, p. 7).
- **Design fields:** IDs, names, status are ordinary modeling decisions. **Latitude/longitude are not required by the brief.**
- **Review:** retain coordinates only if a map, geographic search, routing or location feature is actually in scope. They otherwise introduce data collection/quality expectations without traceable behavior.

LineVariant(id, line_id, direction, valid_from, valid_to) and LineStop(line_variant_id, stop_id, sequence)

- **Source-backed concept:** each line defines an ordered stop sequence and can have schedules (B1, p. 7).
- **Design choices:** separating a logical line from directional/validity variants, adding direction, and using validity dates.
- **Review:** coherent if routes have opposite directions or variants. Since the brief does not state that variants/directions exist, label them as modeling assumptions. LineStop.sequence is the direct representation of ordered stops.

ServiceCalendar(id, days_of_week, date_ranges, exceptions)

- **Source-backed concept:** schedules may vary by day of the week or other operating conditions (B1, p. 7).
- **Design choices:** explicit weekday flags, date ranges and exception records (e.g., holidays/special days).
- **Review:** suitable, but define a service date/time zone and holiday source only if these become necessary. Exact public-holiday semantics are not specified.

Schedule(id, line_variant_id, calendar_id, version, valid_from, valid_to, status), ScheduledTrip(id, schedule_id, service_date, departure_time, vehicle_capacity, status), and StopTime(trip_id, stop_id, sequence, planned_arrival, planned_departure)

- **Source-backed concepts:** schedules, scheduled trips, ordered stops, expected time at each stop and variation in schedules (B1, p. 7); trip capacity (B1, p. 8).
- **Design choices:** version/status/effective-date fields, separate scheduled trip records, sequence values, separate arrival/departure fields, and vehicle capacity assigned at trip level.
- **Review:** largely coherent. Define whether capacity belongs to the trip, vehicle assignment, or a service/vehicle type. Avoid duplicating capacity in both ScheduledTrip.vehicle_capacity and OccupancySnapshot.capacity without defining one as a snapshot/copy and the other as the authoritative value.

Disruption(id, type, scope_type, description, start, end, status, created_by, version), DisruptionTarget(disruption_id, line_id?, stop_id?, trip_id?), DisruptionEffect(disruption_id, delay_minutes?, alt_stops?, cancelled_scope?)

- **Source-backed concepts:** delays, cancellations, diversions, other disruptions; possible line, stop, trip and time scopes; staff management; passenger information reflecting changes (B1, pp. 7–8).
- **Design choices:** normalized target/effect tables, version numbers, description/created-by fields, optional target columns, and detailed effect representation.
- **Review:** suitable as a proposal. Define target constraints so a disruption does not accidentally point to conflicting/missing targets. Clarify whether a target can include multiple lines/stops/trips and how mixed scopes work. alt_stops should preserve order if a diversion is a route change, not merely a set. “Partial cancellation” needs a precise definition.

TicketType(id, name, price, kind[SINGLE|MULTI|PASS|TIMED], validity_period?, max_journeys?, transfer_window?, on_sale, version)

- **Source-backed concepts:** differing prices, validity periods, use conditions and allowed journey counts; staff manage ticket-related information (B1, p. 8).
- **Design choices/assumptions:** the four exact kinds; transfer window; for-sale flag; versioning.
- **Review:** ticket-type flexibility makes sense, but the named kinds and detailed constraints are not requirements in the brief. Prefer a general type plus explicit validity rules until the actual product catalogue is agreed, or mark the kinds as examples. Versioning is useful to prevent a later fare/rule change altering previously sold ticket semantics, but is still a design choice.

Purchase(id, account_id, ticket_type_id, idempotency_key UNIQUE, amount, payment_ref, status, created_at)

- **Source-backed concepts:** passengers acquire priced tickets; duplicate ticket issuance due to communication failures must be avoided (B1, p. 8).
- **Design choices/assumptions:** purchase entity, idempotency key storage, amount/status/timestamps and payment reference.

- **Review:** idempotency protection is well motivated. payment_ref assumes an external or simulated payment process that is not specified. Define whether a purchase can contain one ticket or several; the current one ticket_type_id does not answer quantity/multi-ticket purchases. Ensure a repeated key with different purchase details is rejected rather than silently returning a ticket for the wrong request.

Ticket(id, purchase_id, account_id, ticket_type_id, state, issued_at, activated_at?, expires_at?, journeys_used, version)

- **Source-backed concepts:** ticket association with passenger account; ticket validity based on type, state, previous validations and time; allowed journeys (B1, p. 8).
- **Design choices:** exact states, activation timestamp, expiry timestamp, journey counter, version field and versioned ticket rules.
- **Review:** coherent, but the meaning of activation/expiry depends on unconfirmed ticket policy. Define whether journeys_used counts accepted validations or journeys, especially if transfer/re-validation exists. Ticket type/version should be retained so a rule change does not retrospectively alter issued tickets, if that decision is adopted.

Validation(id, ticket_id, event_id UNIQUE, validator_id, stop_id?, trip_id?, timestamp, result[ACCEPTED|REJECTED], reason_code)

- **Source-backed concepts:** check validity and record successful validations; operational staff need usage/validation activity (B1, p. 8); duplicate validation effects must be avoided after communication failure (B1, p. 8).
- **Design choices:** event ID, validator ID, stop/trip context, timestamp, reason-code vocabulary and database uniqueness.
- **Review:** stable event ID is an effective design for retry safety. Resolve whether rejected attempts are stored in this table; the brief expressly requires successful validation records, while this schema includes rejected records too. If both are persisted, document privacy/retention and reporting behavior. Decide whether rejected requests can reuse an event ID only if the payload is identical.

OccupancySnapshot(trip_id, stop_sequence?, boarded_count, estimated_load, capacity, updated_at)

- **Source-backed concepts:** scheduled-trip occupancy/capacity and occupancy information affected by passenger activity including validation (B1, p. 8).
- **Assumptions/design choices:** boardings as validation-derived count, per-stop snapshot, formula for estimated load, update time and storing capacity with the snapshot.
- **Review:** define the algorithm and whether occupancy is whole-trip, per-stop, or per-segment. If snapshots exist at multiple stop sequences/times, Trip 1..1 OccupancySnapshot is inconsistent; it should be Trip 1..* OccupancySnapshot (or an explicit “current aggregate” table with a single record per trip). Alighting/boarding assumptions must be stated.

Account(id, email, status, created_at)

- **Source-backed concept:** account creation and tickets associated with an account; subscriptions and preferences (B1, p. 8).
- **Design choices:** email as the identifier, status, timestamp and account authentication structure.
- **Review:** email is not specified as the only login identifier. Define the minimum account fields and avoid adding more personal information than needed.

NotificationPreference(account_id, channel, enabled, quiet_hours) and Subscription(id, account_id, target_type, target_id, filters, status)

- **Source-backed concepts:** control subscriptions and notification preferences, subscribe to a line/stop/trip (B1, p. 8).
- **Design choices:** channel rows, enabled flags, quiet-hour representation, filters/status and exact target encoding.
- **Review:** suitable. Channel and quiet-hours support must remain labelled as a detailed interpretation. Define whether preferences are global or per subscription, how multiple channels combine, and what quiet-hours behavior means.

Notification(id, subscription_id, source_event_id, channel, payload, status, created_at, sent_at, dedup_key UNIQUE(source_event_id, subscription_id, channel))

- **Source-backed concepts:** affected subscribers are notified and duplicates must be avoided; notification/history changes over time (B1, p. 8).
- **Design choices:** fields, payload storage, unique dedup key, channel-specific fan-out, created/sent times and status transitions.
- **Review:** dedup key fits the intended behavior. Be precise about event identity: a disruption update/closure may be a new event and therefore legitimately create a new notification. Distinguish the application-level "notification record" guarantee from actual delivery through third-party providers.

StaffUser(id, role) and AuditLog(id, actor, action, entity, before, after, at)

- **Source-backed concept:** operational staff maintain and manage the system (B1, p. 8).
- **Design choices:** a staff-user entity, role taxonomy, audit structure, before/after snapshots and audit timestamps.
- **Review:** sensible if access control/auditing are adopted, but this is not fully specified by the brief. Define staff authentication, allowed operations and audit access only to the extent required by your scope.

7.3 Relationship/cardinality review

The proposed relationships are: Line 1..* LineVariant; LineVariant *..* Stop via LineStop; Schedule 1..* ScheduledTrip 1..* StopTime; Disruption *..* {Line, Stop, Trip}; Ticket 1..* Validation; Trip 1..1 OccupancySnapshot.

- Line to LineVariant and variants to ordered stops are plausible modeling choices; only the line's ordered stop sequence is explicit.

- Schedule/trip/stop-time relationships follow the domain description, although generation and exact cardinalities are design decisions.
- A disruption may affect lines, stops or trips; the many-to-many implementation is a design choice. Confirm whether one disruption can target multiple objects and enforce valid target combinations.
- A ticket may have many validations over time, but exact behavior depends on ticket rules. If only successful validations count for state updates, reflect that in queries and state transitions.
- **Trip 1..1 OccupancySnapshot should be corrected or explained.** Given stop_sequence? and updated_at, several rows per trip seem possible. A clean option is a one-to-many history table plus a separate current aggregate/read model, or a one-to-one current occupancy table without stop-level history.
- The lifecycle table includes **Vehicle type** but the entity list does not. Either remove it or add and justify a corresponding entity and feature; the brief does not require it.

8. Component architecture audit

8.1 Overall style and major compliance point

The file proposes a modular service-oriented architecture, event bus, per-module data ownership, cached read models, asynchronous cross-module effects, and the option to deploy first as a modular monolith.

- **Supported direction:** a distributed, multi-component system, explicit component boundaries/interfaces, synchronous and asynchronous communication, and a separate notification application boundary are required (B2, p. 8; B3, p. 9).
- **Design choices:** event bus, durable at-least-once delivery, separate data ownership, cache/read models, and the particular service decomposition are not prescribed. They are legitimate options requiring justification.
- **Conflict:** if “modular monolith first” means the passenger backend and notification capability are shipped as one deployable, it violates B2’s explicit requirement for independently deployable support and notification functionality across an independent application boundary. A modular monolith can still be used *inside the core backend* while at least one support service (especially notification) is separately deployed. If the brief is interpreted as requiring a supporting service in addition to notification, a second support service could be separated too; clarify this with the teacher rather than assuming.
- **Recommendation:** explicitly document deployment units, not just logical modules. A conservative baseline is (1) passenger web app, (2) core backend, and (3) independently deployed notification service. The notification service is itself a supporting backend component; if the teacher expects an additional service beyond notifications, split a further function (e.g., occupancy or ticketing) rather than relying on ambiguous wording. Do not implement all functionality as one backend REST application: the brief expressly rejects that.

8.2 Architecture diagram components and responsibilities

The current diagram contains Passenger app/Web, Staff back-office, Validator devices, API Gateway, Identity & Account, Network & Schedule, Disruption Service, Ticketing Service, Validation Service, Occupancy Service, Subscription & Notification Service, Event Bus, Passenger Information read model, Audit/History, external notification channels and external payment provider.

- **Passenger app / Web — E.** A passenger-facing web application is expressly required (B2, p. 8). The specific UI stack is a design choice.
- **Staff back-office — P.** Staff must manage network/schedules, ticket-related information and disruptions and see usage/capacity data (B1, p. 8). A dedicated web back-office is a practical design choice, not explicitly mandated as a separate frontend.
- **Validator devices — P/A.** Validation is explicit; physical devices and the deployment location are assumed.
- **API Gateway (authN/Z, rate limit, routing) — D/A.** APIs are required (B2/B3). A gateway, rate limiting and its particular security behavior are not mandated. Ensure the gateway does not become an unhandled single point of failure.
- **Identity & Account Service — P/D.** Account creation, subscriptions and preferences are explicit. Separating identity into an independently deployed service is a choice; a single core backend could own identity if other components remain independently deployable.
- **Network & Schedule Service — E/D.** The domain is explicit; this service boundary is proposed.
- **Disruption Service — E/D.** Disruption management is explicit; separate deployment is a design choice.
- **Passenger Information read model — P/D.** Effective passenger information is required; cached projections/read models are a design choice. State how stale data is indicated and refreshed.
- **Ticketing Service — E/D.** Ticket types/acquisition and account tickets are required; independent deployment is a design choice.
- **Validation Service — E/D.** Validation and recording successful events are required; independent deployment is a design choice. Preserve atomic ticket-state/validation correctness across whichever boundaries are selected.
- **Occupancy Service — E/D.** Occupancy/capacity is required; asynchronous service separation is a design choice.
- **Subscription & Notification Service — E with mandatory boundary.** Subscriptions/notifications are required, and notification capability must cross an independent application boundary (B1, p. 8; B2, p. 8). This component must not be only an in-process module inside one all-in-one deployment.
- **Event bus — D, with good fit.** Asynchronous/event-based communication is required (B3). A particular broker/product and durable at-least-once semantics are choices.
- **Audit / History — P/D.** Historical information is relevant (B1, p. 8); a dedicated append-only service/store is a design choice.

- **External notification channels — A/D.** Notifications are required, but particular external providers/channels are not.
- **External payment provider — A/D.** No external provider is mandated. Keep as an adapter/assumption, not a fixed external dependency.

8.3 Review of each component's data ownership/dependencies

- **Identity & Account:** owns Account and StaffUser; no dependencies is a proposed simplification. Staff identity/roles are not fully specified.
- **Network & Schedule:** owns lines, stops, schedules and trips and publishes events. Domain is source-backed; publishing events is a design response to asynchronous integration.
- **Disruption:** owns disruptions and depends on Network to validate targets. This is a reasonable dependency but should not make disruption creation unavailable whenever Network is temporarily down unless the operation cannot be validated safely.
- **Passenger Information:** owns read models only and depends on events from network, disruption and occupancy. This matches the effective-view concept; cache TTL/staleness semantics are implementation choices.
- **Ticketing:** owns types, purchases and tickets; dependency on payment provider is only relevant if external payment is adopted.
- **Validation:** owns validation records and depends on ticketing data in a same transactional boundary or via a well-defined API. This is the most critical consistency boundary. A separate deployable service must still avoid double-consuming a ticket under concurrent requests.
- **Occupancy:** consumes accepted validation events asynchronously. This improves fault isolation, but does not make a lost-event recovery mechanism automatic; replay/rebuild and deduplication are design responsibilities.
- **Subscription & Notification:** owns subscriptions and notification state, consumes events, resolves recipients, deduplicates and delivers. It should not block core staff/passenger operations; this directly supports the failure-isolation example in B1, p. 8.
- **Audit/History:** consumes events and stores history. This is a useful design but can be combined with another component if the independent-deployment requirements remain satisfied.

8.4 Placement decisions in the requirements file

1. **"Validation and Ticketing share a strong-consistency boundary."** Reasonable design, not a literal brief constraint. The brief does make correctness depend on current ticket state/prior validations and requires duplicate prevention. The implementation must demonstrate race-safe behavior for concurrent validations.
2. **"Occupancy is fed asynchronously by ValidationAccepted events so occupancy failure never blocks boarding."** Sound design that serves the fault-isolation goal. It is not the only compliant solution. Define whether rejected validations contribute to occupancy (normally they should not unless another confirmed rule says so).

3. **"Notification consumes events and never sits in the call path … events accumulate when down."** Well aligned with the explicit notification failure example. The event backlog, storage, retry, and recovery policy are design choices.
4. **Transactional outbox guarantees state change + event atomicity.** Useful design mechanism to prevent lost events, but not specifically required by name in the brief. Tests should cover the crash windows it is intended to prevent.
5. **Gateway timeouts/circuit breakers and a last-known-good Passenger Information cache flagged as possibly outdated.** Timeouts are explicit in B3; circuit breakers/cache/freshness labeling are design choices. Confirm how stale disruptions are represented so users are not misled.

9. Interface and event-catalogue audit

The brief requires defined APIs/messages/events and defined exchanged data (B2/B3). It does not prescribe these paths. The following endpoints are proposed interface designs, not requirements provided by the brief. They are broadly consistent with the functional model; the comments below indicate where the particular choice needs care.

9.1 Passenger information endpoints

- GET /lines — directly maps to FR-1.
- GET /lines/{id}/stops — directly maps to FR-1; the response must preserve stop order.
- GET /stops/{id}/departures?date=&from=&limit= — maps to schedule/trip views, but define whether from is a departure time, cursor or pagination offset and whether results use effective disruption-adjusted values.
- GET /lines/{id}/schedule?date= — maps to FR-2; clarify calendar exceptions and date semantics.
- GET /trips/{id} — maps to FR-3 and may include status/occupancy; clarify stale/unknown occupancy and whether trip data is public.
- GET /disruptions?line=&stop=&active=true — maps to FR-6/FR-7; define active as "now within validity period and current status permits display," or adopt a different explicit rule.

9.2 Account and subscription endpoints

- POST /accounts and POST /auth/login — support FR-19, although the account/authentication design and login mechanism are not in the brief.
- GET/PUT /me/notification-preferences — supports FR-21; define preference granularity, supported channels and quiet-hour behavior.
- POST /me/subscriptions {targetType, targetId, filters} and DELETE /me/subscriptions/{id} — support FR-20/FR-21; define valid target types, ownership, filter schema and whether pause is distinct from delete.
- GET /me/notifications — supports the proposed FR-23; note that FR-23 itself is not explicitly required by the brief.

9.3 Ticket endpoints

- GET /ticket-types — sensible public listing for FR-10/FR-11.

- POST /me/tickets with Idempotency-Key and body {ticketTypeId, paymentToken} — supports ticket acquisition and retry safety. paymentToken presupposes a payment integration; replace it with a provider-neutral payment adapter or mock token if a real external provider is not approved. Define failure, duplicate-key, and mismatched-payload responses.
- GET /me/tickets and GET /me/tickets/{id} — support FR-11 and must enforce account ownership.

9.4 Validation endpoint

- POST /validations {eventId, ticketId|token, validatorId, stopId, tripId?, timestamp} returning 200 {result: ACCEPTED|REJECTED, reason, remaining} — supports FR-13/FR-15. The stable eventId is a design contract supporting retries. Define authentication of a validator, allowed timestamp skew, server/device time trust, ticketId versus token semantics, and behavior when the same eventId arrives with a different payload. These details are not in the brief, but are necessary for a robust implementation.
- Returning HTTP 200 for both accepted and rejected business decisions is a possible API choice; it is not mandated. The API contract should distinguish business rejection from transport/server failures consistently.

9.5 Staff endpoints

- CRUD /admin/lines | stops | schedules | trips plus POST /admin/schedules/{id}/publish — maps to staff maintenance. CRUD and publish are design operations; exact publish/version behavior is not stated in the brief.
- POST /admin/disruptions, PATCH /admin/disruptions/{id}, POST /admin/disruptions/{id}/close — maps to disruption management. Clarify what happens when a closed disruption is later amended and how history is retained.
- CRUD /admin/ticket-types — maps to ticket-related management; precise type/rule schema remains an assumption.
- GET /admin/validations?ticketId=&line=&from=&to= and GET /admin/tickets/{id}/history — supports staff usage/validation information, but not every filter or endpoint is mandated.
- GET /admin/trips/{id}/occupancy — supports capacity/occupancy visibility for staff. Define aggregate vs per-stop semantics.

9.6 Event catalogue

The proposed event catalogue is:

- ScheduleChanged / TripsGenerated — producer: Network; consumers: Passenger Information, Notification, Audit.
- DisruptionCreated / Updated / Closed — producer: Disruption; consumers: Passenger Information, Notification, Audit.
- TicketPurchased, TicketStateChanged — producer: Ticketing; consumers: Audit, and optionally Notification.

- ValidationAccepted / Rejected — producer: Validation; consumers: Occupancy, Audit, analytics.
- OccupancyChanged (thresholds) — producer: Occupancy; consumers: Passenger Information and optionally Notification.

The domain/event subjects are mostly aligned with the brief; event names, consumers, and optional ticket/occupancy notifications are design choices. Ticket-purchase notifications and occupancy-threshold notifications are not specifically required by the brief.

The proposed common event fields—eventId, type, occurredAt, entityId, version, and schema version—are good contract-governance design choices, not source requirements. Consumer deduplication on eventId is an appropriate mechanism to address duplicate/retried messages. Ensure that ScheduleChanged and TripsGenerated semantics are not conflated: a schedule can change without regenerating every trip, and trip generation may be a separate event.

10. Behavioral-model audit

10.1 Ticket lifecycle state machine

The file proposes PURCHASED/ISSUED → ACTIVE → EXPIRED/EXHAUSTED, plus unused expiry and CANCELLED via staff refund/cancel before use, and SUSPENDED for staff/fraud blocks, restorable to ACTIVE.

- **Source-backed:** ticket state and validity affect validation; type, previous validations and time can affect validity (B1, p. 8).
- **Unconfirmed rules:** activated on first valid use, exact issued/active meanings, expiry without use, refunds, cancellation-before-use, suspension/fraud block, restoration and allowed transitions. The brief does not request refunds or fraud tooling explicitly.
- **Recommendation:** present this as a proposed state machine, not a confirmed domain rule. For a minimum scope, agree only the states needed by the chosen ticket types, define time-of-validity semantics, and defer refund/suspension/fraud behavior unless requested. Avoid merging PURCHASED and ISSUED as one displayed state if payment/order confirmation and ticket issuance are separate concepts.

10.2 Validation rules, in order

Current proposal:

1. Duplicate eventId returns stored result.
2. Ticket must exist and be ISSUED or ACTIVE, else INVALID_STATE.
3. Current time must be within [valid_from, expires_at], else EXPIRED or NOT_YET_VALID.
4. For SINGLE, reject further validation unless a transfer window applies; for MULTI, enforce journeys_used < max_journeys and do not consume a journey for a re-validation within a transfer window; for PASS/TIMED, check period and reject a second validation in a short anti-passback interval.

5. On acceptance, insert Validation, update journey count/state with optimistic locking, and write the outbox event in one transaction.

Review: Steps 1–3 and the need to evaluate type/state/prior validations/time are grounded in B1, p. 8; retry/duplicate handling and cross-component consistency align with B1/B3. Exact ticket-kind rules, transfer behavior, anti-passback, interval boundaries, error codes and transaction/outbox implementation are design choices or assumptions. The brief does not give enough information to declare this rule list correct for every allowed ticket type.

Specific risks and fixes:

- Decide whether validity windows are inclusive/exclusive at their endpoints and define the authoritative clock/time zone.
- Clarify whether the validation uses server current time or the client timestamp, and whether offline events can arrive late.
- A transfer window and "single ticket can be validated again" policy are not interchangeable; specify whether a transfer is another boarding, another validation, or a new journey entitlement.
- Anti-passback can reject legitimate consecutive bus rides if the interval is arbitrary; do not include it without ticket-policy approval.
- Two devices validating the same ticket concurrently must not both consume its last journey. Add an explicit concurrency test; optimistic locking is one possible mechanism.
- Keep "successful validations must be recorded" separate from "all rejected attempts are stored." If rejected attempts are stored, say so as an additional decision.
- On retry, return the original *business outcome* for the same event, not a newly computed outcome using a changed ticket state.

10.3 Validation sequence

Proposed sequence: Validator → Gateway → Validation POST /validations(eventId); begin transaction; lookup duplicate event; load Ticket version; evaluate rules; insert Validation, update Ticket where version matches, insert outbox, commit; return result; outbox relay publishes ValidationAccepted to Occupancy and Audit; if the response is lost, retry the same event ID to recover the stored result; retry a version conflict once internally.

- The sequence addresses explicit duplicate-operation and communication-failure concerns (B1, p. 8; B3, p. 9).
- Single-transaction storage and outbox relay are design choices. "Retry a version conflict once" is an arbitrary implementation policy and should be chosen based on transaction behavior, not presented as a brief requirement.
- The diagram should show the rejection path, including whether rejection results are persisted and whether any event is published for a rejected attempt.
- If Validation and Ticketing are separate deployments and do not share a database transaction, the sequence as written cannot simply assume one local ACID transaction spans both services. The document must pick a real consistency protocol/boundary or define a synchronous ticket-consumption API with safe concurrency semantics.

10.4 Disruption-to-notification flow

Proposed flow: staff creates a disruption; Disruption persists and writes an outbox event; Passenger Information updates effective view/cache; Notification resolves affected lines/stops/trips, finds active matching subscriptions, deduplicates by (eventId, subscriptionId, channel), respects preferences/quiet hours, enqueues delivery, retries/backoffs, marks sent/failed and sends to a dead-letter queue after N attempts.

- **Source-backed:** relevant changes must affect passenger information, affected subscribers should be notified, subscriptions/preferences are controllable, and notification failure must not stop unrelated functionality (B1, p. 8).
- **Design choices:** event/outbox, target-resolution algorithm, exact active-filter criteria, dedup key, scheduling/backoff, delivery states and dead-letter queue.
- N attempts is not set and should be a configured policy, not an unstated requirement. Define how an expired notification is treated after retries and whether updates/closures supersede or follow a previous notification.
- Quiet hours defer/suppress behavior is not in the brief; choose and document it if implementing quiet hours.
- The statement that updates/closures create their own events is reasonable because relevant changes and endings need to be reflected. Keep the user-facing decision whether closure itself should trigger a notification explicit.

10.5 Disruption lifecycle

Current proposal: DRAFT → ACTIVE → (UPDATED)* → CLOSED, with SCHEDULED for a future start and automatic EXPIRED after end time; a cancelled trip remains in history and is shown as cancelled, never deleted.

- Disruption periods, managing disruptions and retaining relevant historical information are source-backed (B1, pp. 7–8).
- The named statuses, automatic transition timing and no-delete implementation are design choices. The “never deleted” policy is consistent with historical needs but should be stated as a data-management choice, not a verbatim requirement.
- Consider whether UPDATED is a lasting status or an event/version on an otherwise ACTIVE disruption. A single status field should not have to represent both lifecycle state and “this record was changed.”

10.6 Effective trip computation

Current proposal: start from planned times, apply active disruptions in precedence cancellation > diversion/closure > delay; latest version wins for delays with the same target; output ON_TIME, DELAYED(n), CANCELLED, DIVERTED, adjusted times and occupancy level.

- **Source-backed:** schedules/trips have expected stop times and disruptions affect passenger information (B1, pp. 7–8).

- **Design choice requiring agreement:** precedence, “most recent delay wins,” status enum, how overlapping targets combine, how stop closures affect timing, and whether an occupancy level is included in this response.
- This is exactly the kind of gap the requirements-analysis milestone asks teams to document (B8, p. 4). Keep the proposed precedence as an explicit decision pending confirmation; add examples/tests for overlapping disruption scenarios.

10.7 Degraded modes/failure table

- **Notification down — queue events; browsing and validation continue; delivery resumes:** the key isolation property for notification failure is explicitly stated (B1, p. 8). Durable queued delivery and automatic recovery are design choices but directly serve the requirement.
- **Occupancy down — validation continues; occupancy rebuilt by replaying events:** not stated in these exact terms, but a reasonable application of the general failure-isolation requirement. Event replay requires retained/replayable events and an idempotent rebuild design.
- **Disruption feed stale — serve last-known effective view and mark as possibly outdated:** last-known-good serving and freshness labeling are design choices. The brief says known disruptions should be reflected; serving stale data without an indicator may mislead passengers. Define the maximum tolerated staleness or fail visibly if no reliable view is available.
- **Payment provider down — purchase fails retryably; no ticket without confirmed payment:** wholly depends on the unconfirmed external-payment assumption. If a mock/payment adapter is used, define the simulated equivalent. Do not keep this as a mandatory outage scenario until payment architecture is chosen.
- **Bus or relay down — outbox retains events, none lost:** outbox is a design choice; avoiding unintended effects under communication failures is required. “None lost” is a strong guarantee: define durability/recovery assumptions and demonstrate them in failure tests rather than asserting it without a storage/operations model.

11. Idempotency summary audit

- **Purchase — client idempotency key and unique constraint; retry returns the original ticket. Source support: E/P.** Duplicate ticket issuance after communication failures is explicitly prohibited (B1, p. 8). Good mechanism. Include it if purchase is an operation with retriable requests; decide behavior if the same key is reused with a different payload.
- **Validation — device event ID and unique constraint; retry returns the original result. Source support: E/P.** Duplicate validation recording is explicitly prohibited (B1, p. 8). Good mechanism. The device-generated ID is a design choice; explain how the client preserves it across retries.
- **Notification — dedup key (source_event_id, subscription_id, channel) and provider idempotency key. Source support: E/P/A.** Duplicate notifications must be avoided (B1, p. 8); the exact tuple and provider support are not stated. Good proposed design; label provider idempotency as dependent on provider capability. Define separate logical-notification and external-delivery guarantees.

- **Event processing — consumers persist processed eventIds or use idempotent upserts. Source support: P/D.** Duplicate messages and retries must be handled (B3, p. 9); exact consumer storage strategy is a design choice. Appropriate to at-least-once delivery. Ensure dedup-record changes and side effects are atomic enough to avoid "recorded as processed, effect not applied" failure windows.

12. Design decisions and trade-offs audit

1. **Event-driven decoupling vs immediacy.** Events and asynchronous communication are required for appropriate operations (B3, p. 9); specifically making occupancy/notifications eventually consistent is a design choice. State the freshness delay the design tolerates, if measurable.
2. **Strong consistency only where money/entitlement is at stake.** The principle is a design rationale, not a source quotation. Ticket/validation consistency is well motivated by validity rules and duplicate prevention (B1, p. 8). "Money" depends on whether real payment is in scope.
3. **Computed effective view vs mutated schedules.** The effective view is source-backed; keeping the base schedule unchanged and composing disruptions at read time is a design choice. A cached projection is an optimization, not required. Ensure history remains intelligible.
4. **Occupancy is an estimate; alighting is unobserved; staff correction is allowed.** Occupancy as a domain concept and validation as a possible input are explicit (B1, p. 8); absent alighting observations and staff corrections are assumptions. Keep the estimate caveat and label correction support as optional.
5. **Modular monolith first.** This is the only listed design decision with a direct potential conflict. It is acceptable only if the required independent deployments/boundary remain present. Rewrite the statement to: *"The core backend may be internally modular; the notification capability, and any other service needed to satisfy the supporting-component requirement, must remain independently deployable."*

Review of the five AI-assisted validation suggestions in the original file

The original requirements.md Section 10 proposes five ways to use LLMs/agents to validate the design. These are process recommendations rather than MetroIsJustABus functional requirements, but they align well with the brief's required AI-assisted engineering and evidence collection (B7, p. 10).

1. **Traceability check:** ask a model to map each brief sentence to FR/NFR IDs and identify uncovered statements. **Assessment:** strongly aligned with Milestone 1 traceability and ambiguity documentation (B8, p. 4). The model's mapping must be reviewed by a human; the traceability report should separate direct support from inference, as this report does.
2. **Scenario walk-throughs:** simulate lost validation responses, duplicate purchases, concurrent validation of one ticket, and a disruption closing while a notification is in flight. **Assessment:** appropriate and aligned with duplicate/failure-handling constraints (B1, p. 8; B3, p. 9). Add explicit expected outcomes and automate the highest-

risk cases as integration/system tests; an LLM walkthrough alone is not verification evidence.

3. **Consistency review:** cross-check data model fields against endpoint payloads and event producers/consumers. **Assessment:** useful, and directly supports consistency/traceability. It should be treated as a repeatable check after interface/schema changes. The current file should in particular reconcile the missing ticket-type version field, rejected-validation persistence, occupancy snapshot cardinality and event semantics.
4. **Adversarial review:** ask a model to find single points of failure and race conditions in the component diagram. **Assessment:** useful for fault isolation and concurrent validation, but it cannot prove absence of defects. Pair it with concrete tests for simultaneous last-journey validations, message duplication, provider timeouts, and notification-service outages.
5. **Human review remains necessary for assumptions/open questions.** **Assessment:** exactly right. It should be explicit in the workflow: every assumption or business rule suggested by AI must be accepted/rejected by a human decision owner before it becomes an implementation constraint. Record significant AI suggestions, validation findings, corrections/rework and human decisions for the final evidence/report (B7/B8).

13. Open questions in the current file

The existing seven questions are appropriate and should remain in the project decision log:

1. Are validators connected in real time, or may they work offline (which requires offline rules and later reconciliation)?
2. Is validation per boarding only, or also on exit or transfer?
3. What are the precise transfer-window and anti-passback rules per ticket type?
4. Are refund or ticket transfer operations in scope?
5. Which notification channels are required, and how quickly must they arrive (seconds or minutes)?
6. What are the retention periods for validations and personal data?
7. Does occupancy need per-segment (between-stop) granularity or only per trip?

Additional questions worth adding

8. Must every passenger browse anonymously, and is an authenticated account required to buy tickets?
9. Is a real payment provider required, or is simulated payment acceptable?
10. What is the source of delay/disruption updates: staff, an external feed, vehicle telemetry, or a combination?
11. Which ticket kinds must actually be implemented, and what are the exact validity/activation/journey/transfer rules for each?
12. Are rejected validation attempts persisted, or only returned to the caller? If persisted, for what purpose and how long?

13. What exactly counts as one notification, and is the guarantee about stored records, enqueue attempts, provider acceptance or passenger receipt?
14. Which passenger-facing notifications should be sent for schedule changes, disruption updates/closures, occupancy changes and ticket events?
15. How should conflicting disruptions combine, and what is the precedence rule for cancellation, diversion/closure and delay?
16. Is a modular core plus a separately deployed notification service sufficient for the architecture requirement, or is another independent supporting service expected?
17. What are the approved measurable performance/availability targets, if any?
18. What level of staff-role separation and audit logging is expected?
19. What level of operational stale-data tolerance, deployment rollback/recovery and monitoring evidence will be demonstrated?
20. What is the intended service calendar/time zone and source for holiday/special-day rules?

14. Important brief requirements missing or not made explicit as requirements

This requirements file should not only trace its existing FRs/NFRs; it should also keep a checklist of the brief's mandatory engineering requirements. The following are explicit in the brief and should be tracked in a dedicated "System/engineering constraints" section (they need not be forced into passenger-domain FR numbering).

- **SYS-ARCH-01 — Distributed architecture. Requirement:** implement a distributed multi-component system with a core information system and independently deployable supporting component(s); the notification capability must cross an independent application boundary. A single backend REST application is insufficient. **Evidence:** B2, p. 8. **Note:** critical, particularly because of the modular-monolith note in the current file.
- **SYS-COMM-01 — Both communication modes. Requirement:** use synchronous API communication and asynchronous/event-based communication where appropriate. **Evidence:** B3, p. 9. **Note:** do not make all interactions REST-only.
- **SYS-COMM-02 — Interface contracts. Requirement:** define component interfaces and exchanged data; justify protocols and data formats. **Evidence:** B2/B3, pp. 8–9. **Note:** the endpoint/event catalogue is a good start; add schemas, ownership and compatibility/version rules.
- **SYS-COMM-03 — Communication failure behavior. Requirement:** handle communication failures, timeouts, temporary unavailability, retries, duplicate messages/operations, and cross-component consistency. **Evidence:** B3, p. 9; B1, p. 8. **Note:** a measurable test/scenario checklist is more useful than a generic statement.
- **SYS-TEST-01 — Automated test levels. Requirement:** automate unit, integration, system/end-to-end and relevant acceptance testing. **Evidence:** B4, p. 9; Project Phases, p. 3. **Note:** acceptance testing should not be omitted from the verification plan.
- **SYS-TEST-02 — Interface, failure and quality verification. Requirement:** verify interfaces and interactions between independently deployed components; cover relevant failure/recovery scenarios; include automated code quality and static analysis. **Evidence:** B4, p. 9. **Note:** specify which checks block a release.

- **SYS-CI-01 — Milestone pipeline. Requirement:** build the repository automatically; run automated tests and basic quality checks; ensure successful pipeline execution produces a deployable version and deploys to development. **Evidence:** B8, Milestone 2, p. 4. **Note:** this has a milestone deadline and a 5% evaluation component.
- **SYS-CI-02 — Complete CI/CDE/CD. Requirement:** build/test, create versioned deployable artifacts, validate integrated/system behavior, deploy automatically to development/production, use production gates, and trace source version to artifact to deployed version. **Evidence:** B5, p. 9. **Note:** track independently of domain requirements.
- **SYS-DEP-01 — Separate environments. Requirement:** establish development and production environments; development includes a self-hosted CI/CD runner and supports integration/system tests; production hosts the deployed system. **Evidence:** B6, pp. 9–10. **Note:** explicit infrastructure requirement.
- **SYS-DEP-02 — Containers and reproducible configuration. Requirement:** containerize application components/supporting services and keep reproducible, version-controlled infrastructure/deployment configuration. **Evidence:** B6, pp. 9–10. **Note:** explicit requirement.
- **SYS-DEP-03 — Deployment configuration/recovery. Requirement:** manage configuration, secrets, persistent data, networks and service dependencies; validate deployments and recover from unsuccessful deployments. **Evidence:** B6, p. 10. **Note:** the current degraded-mode section does not fully cover deployment recovery.
- **SYS-OPS-01 — Monitoring/observability. Requirement:** monitor and validate deployed-system behavior; configure monitoring/observability to detect and diagnose failures. **Evidence:** Project Phases, p. 3; B6, pp. 9–10. **Note:** NFR-9 partly covers this, but it should be testable.
- **SYS-AI-01 — AI throughout the lifecycle. Requirement:** use LLMs/agents across requirements, design, implementation, testing, integration, deployment and other appropriate tasks, not only code generation. **Evidence:** B7, p. 10; pp. 1–3. **Note:** explicit course requirement.
- **SYS-AI-02 — Agent context and validation. Requirement:** give agents context and constraints; validate their outputs independently; establish when human intervention is required. **Evidence:** B7, p. 10; pp. 1–3. **Note:** connect this to review/checklist mechanisms.
- **SYS-MEAS-01 — Development evidence. Requirement:** collect sufficient evidence of delegated tasks, agent autonomy, human intervention, validation results, defects, rework, effort and pipeline behavior. **Evidence:** B7, pp. 1–3 and final-report requirements, pp. 4–5. **Note:** operational application metrics and process/AI metrics are different data sets.
- **SYS-TRACE-01 — End-to-end traceability. Requirement:** maintain traceability between requirements, design decisions, tasks, verification, artifacts, source revisions, built artifacts and deployed versions. **Evidence:** B8, Milestone 1 and final-report requirements, pp. 4–5; B5, p. 9. **Note:** link each requirement ID to a design element, test and implementation artifact as work proceeds.
- **SYS-REPORT-01 — Final report and team contributions. Requirement:** report problem analysis and requirements/assumptions; models, architecture, interfaces and

trade-offs; traceability; AI tools/models and agent evolution; testing; CI/CD and deployment/operations; evidence/metrics/critical analysis; each member's main tasks and effort. **Evidence:** B8, final report, pp. 4–5. **Note:** keep the report structure in the project plan so it is not left until the end.

- **SYS-DEMO-01 — Ten-minute final demo. Requirement:** demonstrate passenger functionality, cross-component interactions, relevant normal/failure scenarios, and evidence that the shown system is the deployed version submitted for evaluation. **Evidence:** B8, p. 5. **Note:** plan demonstrable scenarios and deployment-version identity.
- **SYS-DELIVERY-01 — Complete final artifacts and production deployment. Requirement:** submit source code, scripts, configuration, models, docs and artifacts needed to build/test/deploy/operate; complete CI/CDE/CD and deployment config; operational final system deployed in production; process evidence/metrics. **Evidence:** B8, pp. 4–5. **Note:** treat as a release checklist.
- **SYS-COURSE-01 — Evaluation and defense obligations. Requirement:** account for self/peer evaluations, attendance at evaluation periods, defense signup before the deadline, and every member's individual defense and understanding of the complete system. **Evidence:** B8, pp. 2, 4–6. **Note:** these are course obligations, not runtime product requirements, but they should appear in the team plan.

The following are course delivery obligations rather than requirements of MetroIsJustABus' running behavior: teams of five; all members' attendance at evaluation periods; Milestone 1 on 27 October (requirements, assumptions, traceability, component decomposition, and evidence of LLM/agent use); Milestone 2 on 17 November (working build/test/quality/container/development-deploy pipeline); final delivery on 11 December (source and engineering artifacts, deployed production version, AI development evidence, report, 10-minute demo and peer/self-evaluations); defense booking before final deadline and individual defense in which every student must explain the complete system. See B8, pp. 2–6. These are important in the project plan, but they should not be mixed into domain FRs.

15. Recommended revised assumptions/decisions register

The following wording is recommended as a more defensible replacement/addition to the current Section 1. It preserves useful implementation choices while preventing them from being mistaken for facts from the brief.

- **A-01 — Account policy. Proposed wording:** Passengers may browse public network/schedule information without an account. Accounts are required for account-specific tickets, subscriptions and preferences unless stakeholders decide otherwise. **Status:** proposed assumption; confirm purchase access.
- **A-02 — Validation connectivity. Proposed wording:** A validator client submits validation events with a stable event ID. The client may retry the same event after a timeout or lost response. Offline acceptance of new validations is out of scope unless offline rules are explicitly agreed. **Status:** recommended default; avoids conflict with authoritative ticket-state checks.
- **A-03 — Occupancy estimate. Proposed wording:** Accepted validations are an input to estimated trip occupancy, not ground-truth passenger counts. Manual corrections

and sensor integration are optional and not in the baseline until required. **Status:** proposed assumption.

- **A-04 — Vehicle positions/delay source. Proposed wording:** No live vehicle-position/AVL/GPS integration is assumed because no source or interface is specified. Operational staff enter/update disruption or delay information in the baseline. **Status:** proposed scope boundary.
- **A-05 — Payment. Proposed wording:** Ticket acquisition is implemented, but real external payment is not assumed. Use a payment adapter that can be simulated unless a real provider is specifically required. Do not issue a ticket after a failed/uncertain payment result in any actual payment design. **Status:** proposed assumption; confirm.
- **A-06 — Notification channels. Proposed wording:** Notification channels are selected explicitly for the project. The design supports abstraction from the provider, but multiple real channels are not assumed until approved. **Status:** proposed assumption.
- **A-07 — Ticket rules. Proposed wording:** Ticket activation, validity boundaries, journey counting, transfers, anti-passback, cancellation, refunds and suspension remain unresolved rules. Implement only a set approved by stakeholders/teacher. **Status:** open business decision.
- **A-08 — Disruption conflict rules. Proposed wording:** For disruptions, the team will define scope, validity, lifecycle and conflict-precedence rules. Any proposed precedence order is a design decision, not a rule stated by the brief. **Status:** required design decision.
- **A-09 — Staff roles/auditing. Proposed wording:** Staff authorization/auditing will follow a documented minimum policy. Separate planner/dispatcher/ticket-manager roles and detailed before/after audit snapshots are not assumed unless needed and approved. **Status:** proposed security decision.
- **A-10 — Quantitative NFRs. Proposed wording:** Numeric performance/availability targets are not attributed to the brief. Any target the team chooses will be labelled "proposed target," with a measurement method and a test workload. **Status:** mandatory documentation correction.
- **A-11 — Historical retention. Proposed wording:** Historical records will be retained for the period required by the product's operational needs and applicable policy. Exact retention/deletion periods remain to be decided; audit/history and privacy needs must be reconciled. **Status:** open decision.
- **A-12 — Deployment boundaries. Proposed wording:** The core backend may be modular internally, but the notification capability is independently deployable and must communicate across a defined application boundary. Deployment-unit interpretation should be confirmed with the teacher if ambiguous. **Status:** required architecture correction.

16. Prioritized action list

Must fix before the requirements milestone

1. **Fix the modular-monolith conflict** and draw the actual independent deployment boundaries.

2. **Tag every claim** as explicit brief requirement, inference, assumption, or design decision. This can be done with the status labels in this report.
3. **Remove or clearly relabel unsupported NFR targets** (99.9%, p95 values, morning peak).
4. **Separate required behavior from detailed ticket-policy guesses:** transfer, anti-passback, activation, refunds, suspension and exact ticket kinds.
5. **Correct the data-model inconsistencies:** remove or define Vehicle type; correct/explain occupancy snapshot cardinality; distinguish recorded successful validations from optional rejected-attempt logs.
6. **Add the missing system/engineering constraints** from Section 14 to the requirements and verification plan, particularly distributed architecture, communication contracts, test automation, CI/CD, deployment and AI-evidence requirements.
7. **Update the open-question list** to include payment, account policy, event/notification guarantees and disruption conflict rules.

Must decide before implementation of the affected features

8. Whether validators can accept events offline; what constitutes a single journey/transfer; precise ticket lifecycle/time boundaries; which ticket types are in scope.
9. Occupancy granularity/formula and whether manual or sensor corrections exist.
10. Notification channels, quiet-hours behavior, retry limits, deduplication guarantee, and notification types.
11. Disruption effective-view computation when several disruptions overlap.
12. Staff-role/authorization policy and what must be audited.

Can remain design choices if documented and tested

13. REST endpoint paths and status codes; event names/schema fields; API gateway; event broker; outbox and consumer deduplication; cached read model; service subdivision; optimistic locking; retry/backoff and dead-letter handling; specific observability implementation. The team should justify these choices and produce tests/evidence rather than falsely attributing them to the brief.

17. Final assessment

The current file's domain requirements are generally well derived from the technical description. The main issue is **classification and traceability**, not that the proposed design is fundamentally misguided. A few additions are sensible implementation ideas, but the document currently places them beside explicit requirements without a clear distinction. The architecture note about starting as a modular monolith is the most serious compliance risk; the invented numerical NFR targets and unconfirmed ticket rules are the most obvious scope/traceability risks.

For a requirements milestone, the safest version is one that preserves the useful detail in the current file while clearly labelling (1) what the brief explicitly requires, (2) what the team has inferred, (3) what is assumed pending confirmation, and (4) what the team selected as a design.

That separation is itself consistent with the brief, which specifically asks teams to document ambiguities, missing information and assumptions and to maintain traceability.

Appendix A. Compact traceability index

Current file item	Primary evidence / classification
Actors: passengers, operational staff	B1, pp. 7–8 — explicit at general level
Anonymous/registered split, staff subroles	A — not specified in detail
Validator devices and offline state	Validation is explicit; device/location/offline acceptance are assumptions
External notification channels/payment provider	Notifications/acquisition explicit; particular providers/channels are assumptions
Real-time positions out of scope/manual delay input	Proposed scope boundary; not expressly stated
FR-1	B1, p. 7 — E
FR-2	B1, p. 7 — P (exact date/calendar filtering details inferred)
FR-3	B1, p. 7 — E
FR-4	B1, pp. 7–8 — E/P
FR-5	B1, p. 8 — P (retirement/effective-date detail inferred)
FR-6	B1, p. 7–8 — E/P (stop closure subtype inferred)
FR-7	B1, p. 7–8 — P (combined scope/status/lifecycle detail)
FR-8	B1, p. 7 — P/A (effect data and partial-cancellation rules)
FR-9	B1, pp. 7–8 — E/P
FR-10	B1, p. 8 — P (transfer window/on-sale field added)
FR-11	B1, p. 8 — P (registered-only policy inferred)
FR-12	B1, p. 8 — E/P
FR-13	B1, p. 8 — E/P (reason-code representation inferred)
FR-14	B1, p. 8 — E/P (activation/exhaustion states added)
FR-15	B1, p. 8 — E/P (event ID mechanism chosen)
FR-16	B1, p. 8 — P (specific search filters added)
FR-17	B1, p. 8 — P (formula/update mechanics inferred)

Current file item	Primary evidence / classification
FR-18	B1, p. 8 — P (numeric/coarse presentation choices)
FR-19	B1, p. 8 — P (authentication inferred from account function)
FR-20	B1, p. 8 — E/P (filters added)
FR-21	B1, p. 8 — P (quiet hours/pause/cancel details)
FR-22	B1, p. 8 — E/P (dedup identity details)
FR-23	B1, p. 8 — P/A (dedicated passenger history view not explicit)
FR-24	B1, p. 8 — P/A (role authorization/audit structure added)
FR-25	B1, p. 8 — P (specific entities/queryability/retention detail)
NFR-1	B1, p. 8 — P/A; 99.9% unsupported
NFR-2	B1, p. 8; B3, p. 9 — outcome explicit, circuit breakers are D
NFR-3	B1, p. 8 — E/P
NFR-4	B1, p. 8; B3, p. 9 — P/D
NFR-5	A — numerical targets/peak assumptions absent from brief
NFR-6	A/P — proposed security/privacy controls; GDPR not stated in brief
NFR-7	B1, p. 8 — P
NFR-8	A/D — team quality goal, not explicit requirement
NFR-9	B6, p. 3 and pp. 9–10 — P/D
Data model, endpoint paths, event field names and detailed state machines	Primarily D, with underlying concepts traced to B1–B3 as specified in Sections 7–10
CI/CD, tests, deployments, AI process/evidence	B4–B8; add explicit engineering requirements from Section 14